labeled dataset $D_{\text{probe}} = \{x_{jt}, z_{jt}\}$ which provides labels for a subset of the tokens (e.g., the tense of every verb). In our setting, we focus on the MLP neuron activations $a^{(\ell)} = \sigma(W_{fc}^{(l)}\gamma(h_t^{(l-1)}))$—the set of representations immediately after the elementwise nonlinearity—and train a linear binary classifier $g_\ell(a_{jt}^{(\ell)}) = \hat{z}_{jt}$ to minimize a classification loss $\mathcal{L}(z_{jt}, \hat{z}_{jt})$ for each layer of the network.

Key Concepts We make frequent reference to the concept of a *feature*. The field has not yet settled on a consensus definition [4], but for our purposes, we mean an interpretable property of the input that would be recognizable to most humans. A *monosemantic* neuron is a neuron which activates for exactly one feature, though this can also be subtle depending on what one is willing to count as one feature versus a composition of related features. In contrast, a *polysemantic* neuron is a neuron which activates for multiple unrelated features. Polysemanticity is a consequence of *superposition*, the phenomenon of representing n features with $d < n$ dimensions. We focus on applying sparse probing to the activations of the MLP layers because these activations form a *privileged basis* [41]. That is, because applying an elementwise nonlinearity breaks a rotational invariance of the representations, it is more likely that the basis dimensions (each neuron) are independently meaningful. Without a privileged basis, as in the residual stream, there is no reason for features to be basis aligned, and therefore no reason to expect sparsity (modulo optimization quirks [66]). Finally, superposition manifests differently in the residual stream than in the neuron activations because of this privileged basis; we discuss this further in A.9.

3.2 Sparse Feature Selection Methods

Rather than just localize a feature to a particular layer, we attempt to identify a single neuron or a sparse subset of neurons which fire if (and ideally only if) the feature is present in the input. This more granular localization can be accomplished by training a k-sparse probe, a linear classifier with at most k non-zero coefficients. The problem becomes, which of the $\binom{d}{k}$ possible neuron subsets are most predictive? While this is an $\mathcal{NP}$-hard combinatorial optimization problem, there exist a number of fast heuristics and tractable optimal algorithms [67].

In probing, this is typically formalized as a ranking problem [13], where neurons are individually scored by some importance measure and then rank ordered to include the top k. Given a binary classification dataset, natural scoring rules that have been explored are the absolute difference between class means for each neuron [13], the mutual information between each neuron and the labels [68, 32, 33], or the absolute magnitude of the coefficients of a dense probe trained with l_1 regularization [27].

We propose two additional techniques: adaptive thresholding and optimal sparse probing (OSP). OSP leverages recent advantages in sparse prediction solution techniques to train a k-sparse classifier to provable optimality using a tractable cutting plane algorithm [17], though this is only feasible for smaller values of k. For larger ranges of k, we use adaptive thresholding to train a series of classifiers that iteratively decrease the value of k, where at each step t we retrain a probe to only use the top k_t neurons with highest coefficient magnitude from the previous k_{t-1}-sparse probe. For the sake of computational efficiency, in both methods we perform an initial filtering step to take the top neurons with maximum mean difference.

For all methods, we retrain a logistic regression probe for the k neurons selected. Additionally, while we focus on classification, all of these methods have a straightforward generalization to continuous targets.

3.3 Probing in Practice

Constructing Probe Datasets An informative probing experiment begins with designing a suitable probing dataset. To illustrate a common issue, take the case of probing for the `is_politician` feature. If the dataset just includes the names of people labeled as politicians or not, the trained probe will not be able to distinguish the model's `is_politician` feature from the `is_political` feature which fires for all political content (e.g., "The Democratic Party"). On the other extreme, if the dataset contains just the names of politicians and random tokens, the probe won't distinguish `is_politician` from `is_person` as very few random tokens concern non-politician people. In other words, there is a general tension in how to shape the negative examples in the dataset to create the most *conceptual separation* between the true feature of interest and all possible correlates.

Another issue arises when the feature of interest is a property of multiple tokens or a full sequence of tokens (e.g., `is_python_comment` or `is_french`). If the relevant neuron fires on every token in the feature sequence, one could just sample tokens randomly, but there might be more specific conditions on when a neuron fires that cannot be known *a*

priori[2]. A solution to this is to perform an elementwise aggregation (e.g., mean or max) of the activations over the token span of each occurrence of the feature. For example, for a dataset of sequences in many different languages, the input to our probe could be the average activation vector for each sequence, with the target being the language of the sequence. Unfortunately, this gives a somewhat weaker result, as this process is not able to distinguish the `is_french_noun` feature from the `is_french_verb` feature and thus requires further analysis to interpret correctly.

There are additional subtleties having to do with feature granularity, rare features, and overlapping features [15] which we explore in Section 5.4. For all of these issues, the most appropriate recourse will depend on the researcher's intent and the nature of the feature being probed for.

Evaluation and Interpretation To evaluate the performance of a probe, we compute the number of true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN) of our binary classifier on an out-of-sample test set. We then calculate the precision (PR), recall (RE), and F1 score (F1):

$$\mathrm{PR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \qquad \mathrm{RE} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \qquad \mathrm{F1} = \frac{2\mathrm{PR} \times \mathrm{RE}}{\mathrm{PR} + \mathrm{RE}}$$

We use the F1 score as our primary evaluation metrics to determine which neurons are most likely associated with the target feature given the asymmetric importance of the positive class[3]. Precision and recall give insight into how the selected neurons' implicit feature granularity compares to the feature being probed for. Low precision and high recall indicates either selected neurons are highly polysemantic or the model represents a more general feature than is being probed for. High precision and low recall of the probing classifier may indicate that the identified submodule represents a more specific feature than the feature being probed for (e.g. `is_french_noun` instead of `is_french`). We can adjust our preference for classifiers with high precision or high recall by modifying the threshold of the classifier or by tuning the class weights in the probe loss function; a higher class weight for the positive class (assuming it has fewer samples than the negative class) will prioritize recall over precision.

4 Empirical Overview

Models We study EleutherAI's Pythia suite of autoregressive transformer language models [70] [4]. These models are fairly standard GPT variants that use parallel attention and rotary positional encodings and were trained on The Pile [71]. In particular, we run experiments on the 7 models ranging from 70M to 6.9B parameters; full model hyperparameters are given in Table 1.

Data We study ten different feature collections: the natural language of Europarl documents, the programming language of Github source files, the data source of documents from The Pile, the part-of-speech and grammatical dependency of individual tokens, morphological features of tokens (e.g., verb tense), plain text features of tokens (e.g., whitespace or capitalization), the presence of specific compound words, LaTeX features in ArXiv documents, and a number of factual features associated with people (e.g., gender, occupation). Full descriptions of datasets, their construction process, and summary statistics are available in B.2. All categorical features are converted into separate one-versus-all binary features for over 100 total binary classification tasks.

Experiments For each combination of model, feature, layer, and sparse feature selection methods, we train probes for a range of values for k and report classification performance on a held-out test set. We compare the feature selection methods in B.5 and illustrate the relationship between model size and sparsity in Figure 7. For neurons identified as being especially relevant, we save the activations over larger text data sets and perform further analyses described throughout Section 5. All code and data are available at `https://github.com/wesg52/sparse-probing-paper`.

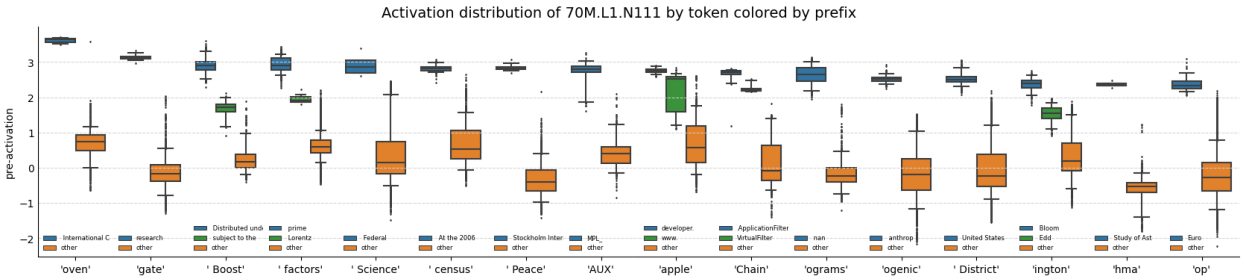
5 Case Studies

We conduct a series of more detailed case studies to carefully study the behavior of individual neurons, while also illustrating the challenges that pose barriers to further progress. Although we zoom in on very narrow examples, we found many neurons of the same category in our probing experiments and believe these examples are representative of broader neuron families [5] that exist in all LLMs.

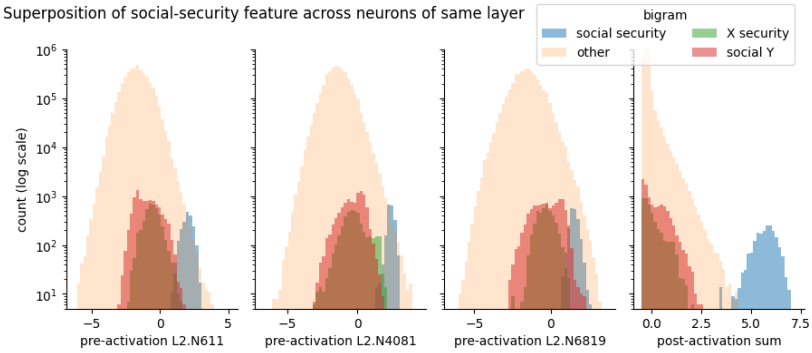
[2]As an example, in the case of factual recall, the results from [69] suggest that MLP neurons are especially important for the last token of a multi-token subject.

[3]We also measure Matthew's Correlation Coefficient (MCC), a balanced accuracy metric, and the results are largely the same.

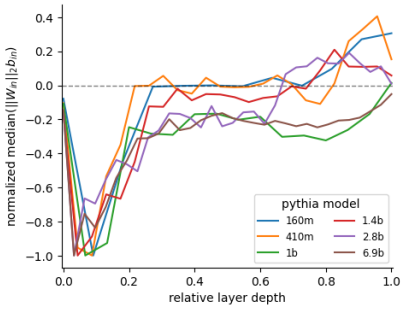
[4]`https://github.com/EleutherAI/pythia`; unfortunately, our experiments were performed with the `V0` suite of models which have recently been updated.



(a) Activations of a single polysemantic neuron on different tokens when preceded by specific stimuli.



(b) Example of superposition in representing the compound word "social security."



(c) Evidence of superposition in early layers

Figure 2: Summary of our key results of superposition. Polysemantic neurons firing on many diverse stimuli (a) are necessary to implement superposition (b) to adequately represent many more possible n-grams than dimensions. The natural mechanism for implementing superposition in toy models leverages large weight norms and negative biases[4] (see Section C for additional motivation and discussion), which we observe to be most associated with early layers (c).

5.1 Superposition in the Wild: Compound Word Neurons

The token vocabulary is a fairly unnatural symbolic language to perform all of the linguistic and conceptual processing required of a language model. For instance, compound words like "social security" are treated as two separate tokens, despite meaning something very different when the tokens appear together versus apart. Moreover, quirks of spacing and capitalization frequently cause words to be broken up (e.g., despite " Harvard" being a single token, when not preceded by a space "Harvard" gets tokenized into "Har" and "vard"). Based on many such examples, [10] hypothesize that a primary function of the early layer neurons is to "de-tokenize" the raw tokens into a sequence of more useful abstractions. However, this *pseudo-vocabulary* might be extremely large (e.g., all common n-grams) making it a natural candidate for being represented in superposition. To investigate this, we probe for neurons which respond to 21 different compound words, where the first and second words mean something quite different when appearing separately versus together (e.g., "prime factors").

After probing for neurons which activate for specific compound words `XY` while not firing on any bigrams of the form `XZ` or `WY` $\forall$ `W,Z`$\in \mathcal{V}$, `Z`$\neq$`Y`, `W`$\neq$`X` for each compound word we found *many* individual neurons which were almost perfectly discriminating. However, after inspecting the activations across a much wider text corpus, we observe these neurons activate for a huge variety of unrelated n-grams (see Figure 2(a)), a classic example of the well known phenomena of polysemanticity [5, 12, 10]. Superposition implies polysemanticity by the pigeonhole principle—if a layer of n neurons reacts to a set of $m \gg n$ features, then neurons must (on average) respond to more than one feature. This example also underscores the dangers of "interpretability illusions" caused by interpreting neurons using just the maximum activating dataset examples [72]. A researcher who just looked at the top 20 activating examples would be blind to all of the additional complexity of neuron `70M.L1.N111`, with Figure 2(a) still only scratching the surface[5]. While inconvenient for interpretability researchers, polysemanticity is also problematic for the model, as it causes interference between

[5]This figure was generated with 300 million tokens from the Pile Test set but when run on 10 billion tokens of the Pile train set 16 of the top 20 dataset examples show the neuron activating on the "he" of German words starting with "Schönhe". These maximum activating dataset examples can be viewed on Neuroscope[73]